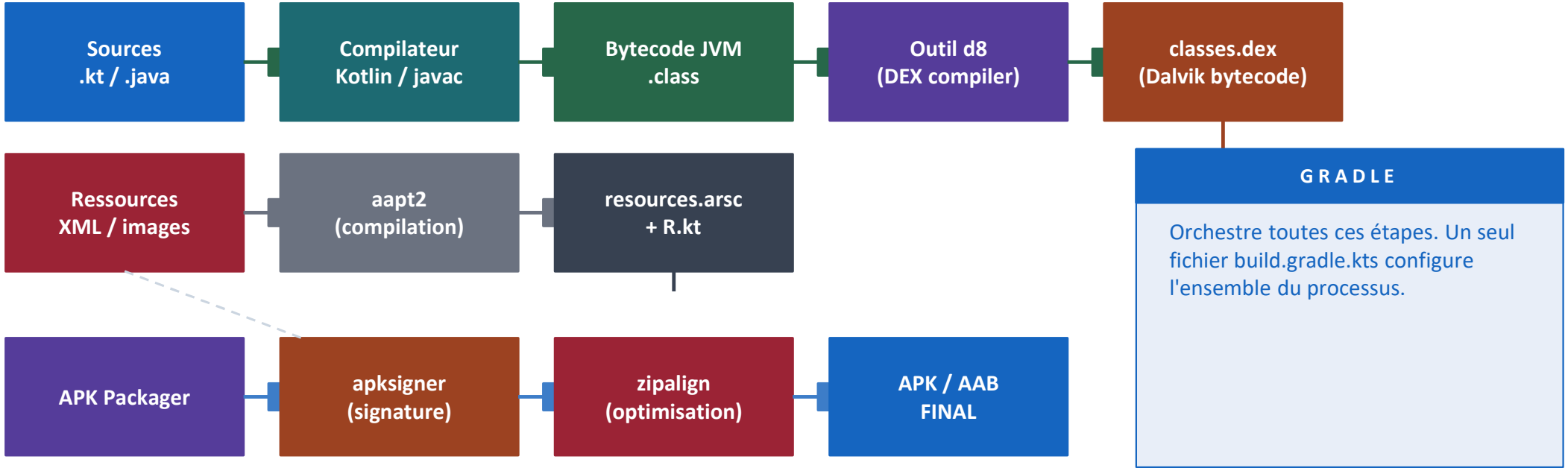
II

Processus de Construction

De la source Kotlin/Java au fichier APK/AAB signé

Schéma complet du pipeline de build

7 étapes orchestrées par Gradle, du code source à l'APK déployable.



Gradle — Système de build officiel

Gradle lit la configuration et déclenche automatiquement chaque outil dans le bon ordre.

Rôle de Gradle

Lit build.gradle.kts et orchestre tout : téléchargement des dépendances, compilation, génération R.kt, conversion DEX, assemblage et signature. Sans Gradle, chaque étape devrait être lancée manuellement.

Variantes de build

debug : logs activés, signatures automatiques, R8 désactivé. Installation directe possible.
release : R8 actif (minification + obfuscation), signature avec le keystore officiel. Version publiée sur le Play Store.

R8 — Minification et obfuscation

R8 remplace ProGuard depuis AGP 3.4. En une passe : suppression du code mort (tree-shaking), renommage des classes (obfuscation) et optimisation du bytecode. Réduit l'APK de 30 à 60 %.

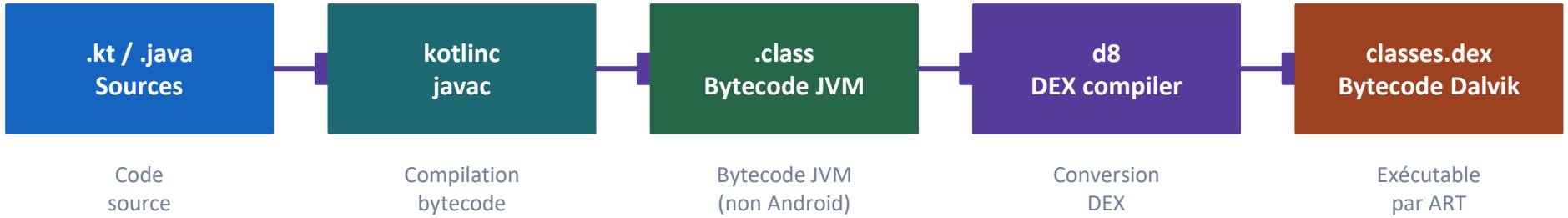
Dépendances Maven

Déclarées par coordonnées (groupe:artefact:version). Gradle les résout depuis Maven Central ou JitPack. Dépendances transitives résolues et fusionnées automatiquement.

```
android { compileSdk=35; defaultConfig { applicationId="com.ex.app"; minSdk=26; targetSdk=35 }
  buildTypes { release { isMinifyEnabled=true; isShrinkResources=true } } }
dependencies { implementation("androidx.core:core-ktx:1.13.1") }
```

Compilation DEX

Le bytecode JVM (.class) n'est pas exécutable sur Android. L'outil d8 le convertit au format Dalvik (.dex).



Pourquoi un bytecode spécifique ?

La JVM utilise une architecture à pile d'opérandes. ART utilise une architecture à registres, plus efficace pour les contraintes mobiles : moins d'instructions, meilleure gestion de la mémoire et de la batterie.

Multidex

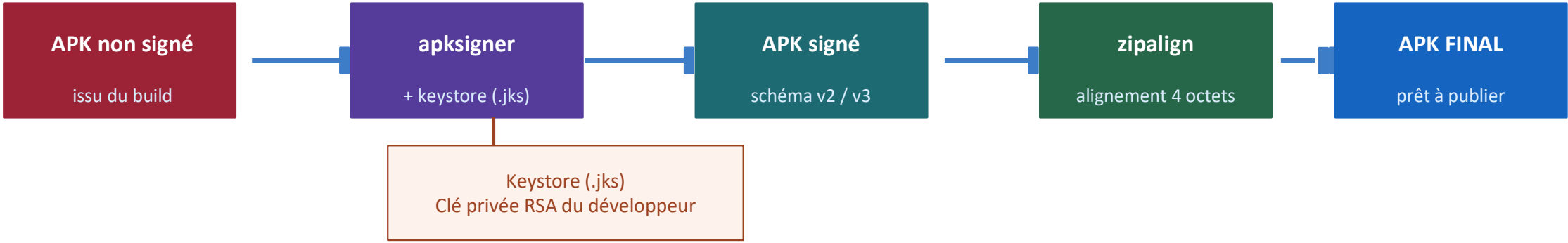
Un fichier DEX supporte au maximum 65 536 méthodes. Les apps utilisant de nombreuses bibliothèques dépassent cette limite. Gradle génère alors classes.dex, classes2.dex… ART les charge tous au démarrage.

aapt2 — Ressources

En parallèle du code, aapt2 compile les XML (layouts, strings) en format binaire compact (resources.arsc). Il génère aussi R.kt contenant les identifiants permettant d'accéder aux ressources depuis le code Kotlin/Java.

Signature de l'APK — Processus et schéma

Tout APK doit être signé numériquement pour être installé ou publié sur le Play Store.



Pourquoi signer ?

La signature prouve l'identité de l'auteur : seul le détenteur de la clé privée peut signer. Android et le Play Store rejettent tout APK non signé. Toute modification après signature invalide celle-ci.

Schémas v1 à v4

v1 : signature ZIP héritée de Java.
v2 (Android 7+) : signature de l'APK entier. Recommandé.
v3 (Android 9+) : rotation de clés possible.
v4 (Android 11+) : streaming APK optimisé.

En pratique : signer v2+v3 couvre 99 % des appareils.

zipalign — Optimisation mémoire

zipalign réorganise les données non compressées sur des frontières de 4 octets. Le système accède alors aux ressources directement en mémoire mappée (mmap), sans copie.

Résultat : démarrage plus rapide, RAM réduite. Obligatoire avant le Play Store.

Distribution & installation des applications

Une application Android est empaquetée puis distribuée. Deux formats coexistent :

APK — Android Package (format historique)

Archive ZIP contenant tout : code DEX, ressources pour toutes densités et langues, bibliothèques natives.

Avantage : déploiement direct.
Inconvénient : taille non optimisée.

Usage actuel : tests internes, distribution hors Play Store.

AAB — Android App Bundle ★ Recommandé

Format intelligent, obligatoire sur Play Store depuis août 2021.

Google Play Dynamic Delivery génère des APK sur mesure par appareil : bonne densité, langue et architecture CPU uniquement.

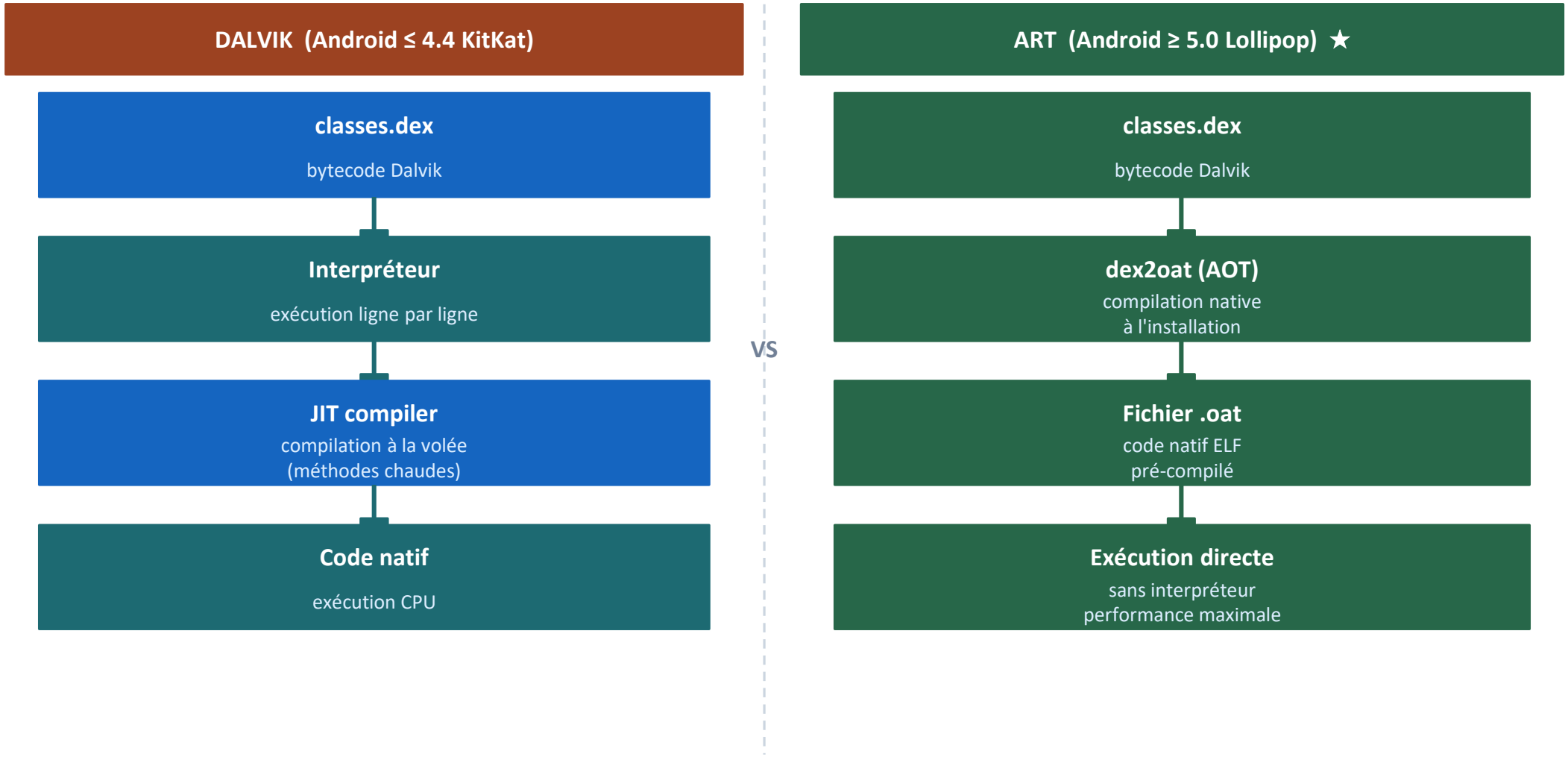
Réduction de taille : 10 à 15 % par utilisateur.

Canaux d'installation	
Google Play Store	Canal principal. Compte développeur requis (25 $). Revue automatique + manuelle. 2,5 milliards d'appareils actifs.
F-Droid	Boutique alternative libre et open-source. Aucun traqueur accepté. Populaire chez les utilisateurs soucieux de leur vie privée.
Sideloading	Via ADB (adb install app.apk) ou gestionnaire de fichiers. Requiert l'activation des « Sources inconnues ».

III Android Runtime (ART)

De Dalvik à ART : évolution du moteur d'exécution

Dalvik vs ART — Schéma comparatif



ART — Stratégie hybride AOT + JIT (Android 7+)

Depuis Android 7.0 (Nougat), ART combine AOT, JIT et profilage pour concilier démarrage rapide et performances optimales.

1

Première installation

JIT actif. Démarrage immédiat sans AOT complète. Le JIT profile l'exécution et collecte des statistiques.

2

Utilisation courante

Le JIT identifie les méthodes chaudes (hot methods) et enregistre un profil (.prof) lié aux habitudes réelles de l'utilisateur.

3

Appareil en veille (secteur)

Le démon dex2oat compile AOT uniquement les méthodes du profil, en arrière-plan, de façon transparente.

4

Lancements suivants

Méthodes critiques en code natif pré-compilé. Démarrage et exécution nettement plus rapides.

✓ Démarrage rapide ✓ Performances optimales ✓ GC amélioré ✓ Autonomie préservée

⚠ Installation légèrement plus longue ⚠ Espace disque additionnel

IV

Structure APK & AndroidManifest

Anatomie d'une application Android installée

Structure interne d'un fichier APK

Un APK est une archive ZIP. Explorer avec : unzip monapp.apk -d dossier/

AndroidManifest.xml

Composants, permissions, métadonnées. Premier fichier lu par Android à l'installation. Obligatoire.

classes.dex (+ classes2...)

Bytecode Dalvik exécuté par ART. Multidex si > 65 536 méthodes.

resources.arsc

Table binaire compacte : strings, dimensions, couleurs. Chargée en mémoire au démarrage.

res/

Images PNG/WebP, XML layouts. Extraits et utilisés à l'exécution.

lib/

Bibliothèques .so par ABI : arm64-v8a, armeabi-v7a, x86, x86_64.

META-INF/

Signature : CERT.RSA, CERT.SF, MANIFEST.MF. Garantit l'intégrité de l'APK.

AndroidManifest.xml — Rôle et structure

Le Manifest est le passeport de l'application auprès du système Android. Sans lui, l'application ne peut pas être installée.

Déclaration des composants

Chaque Activity, Service, BroadcastReceiver et ContentProvider doit être déclaré. Un composant non déclaré est invisible du système. C'est la carte d'identité de chaque composant de l'application.

Permissions

Déclaration des permissions requises : Internet, caméra, contacts, localisation… Depuis Android 6, les permissions dangereuses doivent être demandées à l'exécution, au moment précis où elles sont nécessaires.

Version minimale du SDK

minSdk : version Android minimale supportée. L'app sera invisible sur Play Store pour les appareils inférieurs.
targetSdk : version pour laquelle l'app est optimisée. Toujours aligner sur la dernière API stable.

Fonctionnalités matérielles

La balise <uses-feature> déclare les composants matériels indispensables. Exemples : caméra arrière (scanner QR), NFC (paiement), GPS (navigation).

Play Store exclut automatiquement les appareils incompatibles.

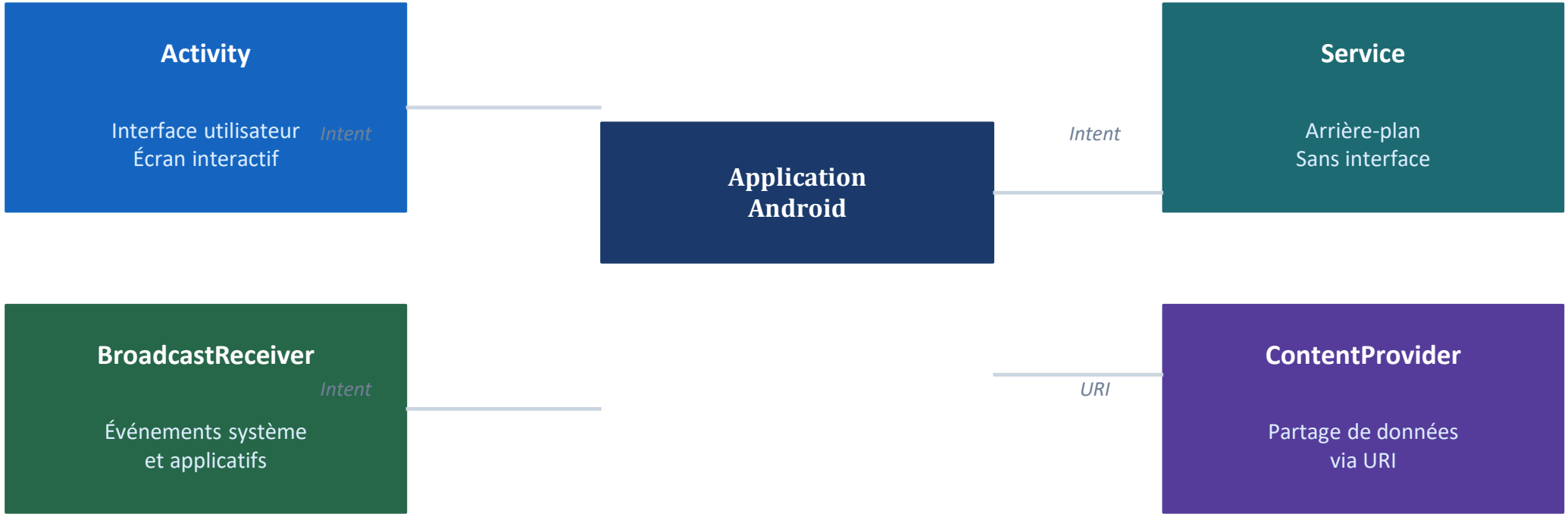
V

Composants d'Application Android

Les quatre briques fondamentales de toute app Android

Les quatre composants fondamentaux

Particularité Android : il n'existe pas de main(). L'application est un ensemble de composants indépendants, chacun pouvant être lancé par le système ou une autre application via des Intents.



WhatsApp : liste conversations, chat, appel vidéo

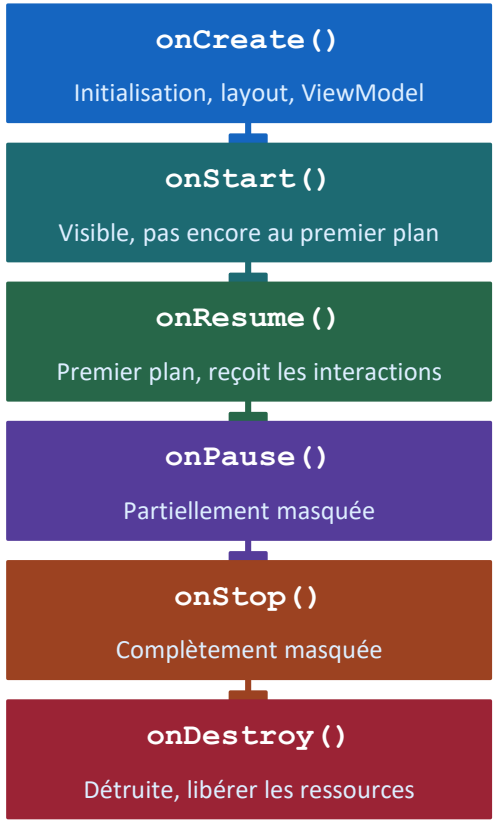
Spotify : lecture, synchronisation arrière-plan

Réception SMS, connexion Wi-Fi, démarrage appareil

Contacts (ContactsContract), Galerie (MediaStore)

Activity — Cycle de vie

Une Activity représente un écran. Elle possède un cycle de vie précis géré par le système Android.



Exemple concret : WhatsApp

Pendant une conversation, l'utilisateur reçoit un appel : la ConversationActivity passe en onPause() (dialogue d'appel). Si elle accepte l'appel : onStop(). Quand l'appel se termine : onStart() → onResume().

Rotation d'écran — Piège classique

Une rotation détruit et recrée l'Activity (onDestroy → onCreate). Les données non sauvegardées sont perdues. Solution : ViewModel (survit à la rotation) + onSaveInstanceState pour les petites données UI.

Intent — Navigation entre Activities

Pour naviguer entre écrans, on crée un Intent ciblant l'Activity souhaitée, avec des données en extra (putExtra). L'Activity destination les récupère dans son onCreate().

Service — Traitement en arrière-plan

Un Service effectue des opérations de longue durée sans interface graphique, même si l'utilisateur quitte l'application.

Foreground Service	Background Service	Bound Service
Notification permanente obligatoire (Android 8+)	Invisible, très limité depuis Android 8	Lié à un client via IBinder
Spotify lecteur | GPS Maps | Appel actif	*Synchronisation données → WorkManager recommandé*	*Service GPS partagé* *Service paiement NFC*

WorkManager — Solution recommandée pour les tâches planifiées

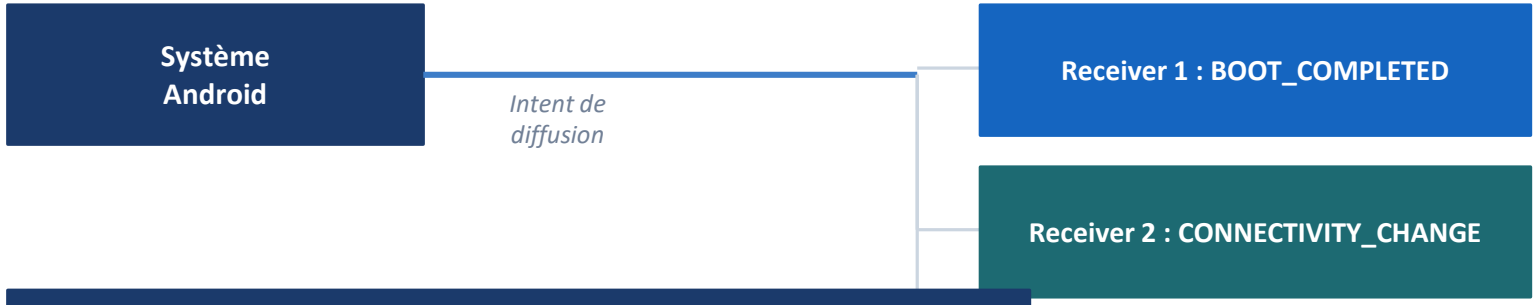
Bibliothèque Jetpack pour toute tâche devant s'exécuter de façon garantie en arrière-plan, même si l'app est fermée ou si l'appareil redémarre.

Gère les contraintes automatiquement : réseau disponible, batterie suffisante, stockage disponible. Respecte Doze Mode et App Standby.

Exemples : upload photos (Google Photos), synchronisation e-mails, compression fichiers, rapports analytiques.

BroadcastReceiver — Écoute d'événements

Un BroadcastReceiver écoute et réagit à des annonces (Intents de diffusion) du système ou d'autres applications.



Événements système	
BOOT_COMPLETED	Lancement au démarrage
CONNECTIVITY_CHANGE	Wi-Fi/mobile dispo
BATTERY_LOW	Batterie faible
AIRPLANE_MODE	Mode avion activé

Événements applicatifs	
Notification FCM	Push Firebase Cloud Messaging
DownloadManager	Téléchargement terminé
AlarmManager	Alarme déclenchée
Intent personnalisé	D'une autre application

Restriction Android 8+ : la plupart des BroadcastReceivers implicites en arrière-plan sont interdits. Utiliser FCM pour les notifications push, et WorkManager pour les tâches planifiées.

Content Provider — Partage de données

Un Content Provider expose des données structurées à d'autres applications via une interface adressée par des URI.

`content://com.android.contacts/contacts`

Content Providers système intégrés

ContactsContract : contacts téléphoniques (WhatsApp, Gmail, composeur).
MediaStore : photos, vidéos, musique (Galerie, Spotify).
CalendarContract : agenda (Google Calendar, Outlook).
CallLog.Calls : historique des appels.

Opérations disponibles

query() : lire les données (Cursor).
insert() : ajouter un enregistrement.
update() : modifier les données.
delete() : supprimer des enregistrements.
Ces opérations correspondent aux opérations SQL.

Sécurité d'accès

L'accès est contrôlé par des permissions (READ_CONTACTS, WRITE_CONTACTS…). L'utilisateur doit les accorder explicitement.

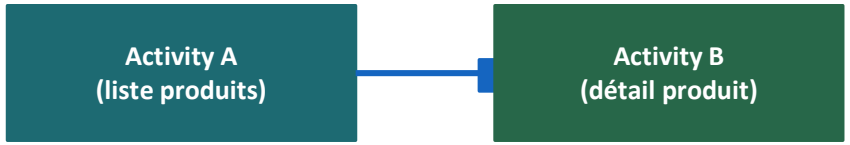
Quand créer le sien ?

Utile pour partager avec d'autres apps ou un widget. Pour usage interne uniquement : Room (ORM Jetpack) est plus simple.

Intents — Communication entre composants

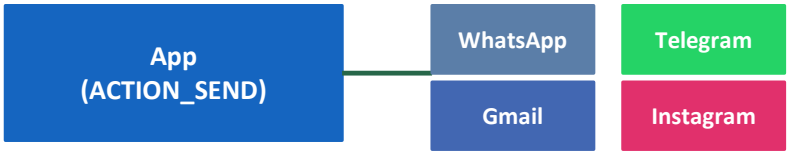
L'Intent est le mécanisme universel d'Android pour déclencher des composants, dans la même app ou entre applications.

Intent Explicite — Navigation interne




```
Intent(this, DetailActivity::class.java) +
putExtra("id", produitId)
```

Intent Implicite — Action ouverte



Android choisit l'app

PendingIntent — Intent différé

Enveloppe un Intent à exécuter ultérieurement par le système (AlarmManager, notifications). Boutons de notification, alarmes, widgets. Android 12+ : FLAG_IMMUTABLE obligatoire.

Intent Flags importants

FLAG_ACTIVITY_CLEAR_TOP : dépile les Activities au-dessus.
FLAG_ACTIVITY_SINGLE_TOP : réutilise l'instance existante.
FLAG_ACTIVITY_NO_HISTORY : ne s'ajoute pas au back stack.

Sécurité Android 12+ : les Intents implicites vers une autre application ne fonctionnent que si le composant cible déclare android:exported="true" dans son Manifest. Sans cette déclaration, le système bloque l'Intent pour prévenir les accès non autorisés.